MoonAP v0.1: A MoonBit Agent Playground for LLM-Generated Browser-Local WebAssembly Applications

Maomao Tang
CUHK-SZ
maomaotang@link.cuhk.edu.cn

April 2026

Abstract

Large language models (LLMs) can synthesize useful programs from natural-language descriptions, but source code alone is not yet a complete application experience. A practical software synthesis environment must compile generated code, provide inputs, execute safely, present results, preserve useful artifacts, and avoid unnecessary movement of private data. This paper presents MoonAP v0.1, a MoonBit-oriented agent playground that turns a natural-language request into MoonBit source code, compiles it to WebAssembly, runs it locally in the browser, and saves successful applications as reusable MoonAP-SKILL packages. The system combines an LLM routing layer, a MoonBit compilation service, declarative runtime contracts, browser-local file execution, and a three-layer SKILL architecture for personal reuse, local installation, and cloud discovery. The v0.1 prototype demonstrates twelve task categories, from calculators and JSON/text utilities to CSV analysis and large FastQ generation/analysis. Its main contribution is a platform pattern for LLM-generated applications where raw user files remain on the client, browser-side computation handles large inputs, and saved SKILLs can be rerun without repeated LLM calls. We also report a pragmatic lesson from targeting a young language: when real lightweight models do not yet generate MoonBit reliably, deterministic simulation paths, explicit runtime contracts, and artifact reuse are essential for building and evaluating the surrounding synthesis system.

1 Introduction

LLM-based software synthesis has made it possible for users to describe a program in natural language and receive source code [2, 1, 3]. Most benchmarks and demonstrations emphasize the generation step: whether the model can produce a correct function, solve a programming problem, or pass tests. In everyday use, however, generated source code is only one part of the path to a working application. A practical system must select or configure a model, preserve prompts and generated source, compile the program, expose runtime inputs, execute the artifact, present results, and allow successful programs to be reused. These requirements become more pronounced when the target language is new, when the generated program is intended to run in a browser, or when user data should remain local.

MoonAP is a MoonBit Agent Playground designed to explore this complete workflow. A user describes an application, the system asks an LLM to generate MoonBit code, a local server compiles the code to WebAssembly, and the browser runs the resulting application with a task-specific runtime UI. If the result is useful, the application can be saved as a MoonAP-SKILL and reused later without another LLM call. In this model, the LLM is used to create or repair an application, while routine execution shifts to the user's machine.

MoonBit is a young programming language with a toolchain oriented toward modern application targets, including WebAssembly [5, 8]. This makes it attractive for browser-local synthesized

applications, but it also exposes a difficulty: many lightweight or free LLM routes do not yet generate valid MoonBit code consistently. MoonAP therefore treats code generation as only one part of the system. It pairs real API routes with deterministic simulated routes, embeds runtime specifications in generated artifacts, and separates generation-time behavior from reuse-time behavior.

This paper reports on MoonAP v0.1 as a system prototype. The contribution is a working architecture for connecting LLM-generated MoonBit code to browser-local execution and reuse:

- an end-to-end pipeline from natural-language task to MoonBit source, WebAssembly compilation, browser-local execution, and reusable SKILL;
- a runtime contract abstraction for generated applications, covering form, text, report, file, and large-file workflows;
- a three-layer SKILL model that distinguishes personal reuse, locally installed public SKILLs, and cloud discovery;
- browser-local large-file workflows for FastQ generation and analysis, including streamed and chunked processing paths designed for GB-scale files whose raw contents remain on the client;
- a data-local execution model in which raw files are processed near the user and successful applications can be rerun without repeated generation;
- an implementation and qualitative evaluation over twelve v0.1 demonstration categories.

2 Related Work and Positioning

LLM program synthesis. Prior work has shown that large language models can synthesize code from natural-language prompts and solve programming tasks at increasing levels of difficulty [2, 1, 3]. MoonAP builds on this line of work but studies a different part of the lifecycle. Instead of asking only whether a model can emit correct source code, MoonAP asks how generated code becomes a runnable, inspectable, reusable, and locally executed application.

MoonBit and WebAssembly. MoonBit targets modern software delivery scenarios, and WebAssembly provides a portable execution target for browser and non-browser environments [5, 8]. This combination is well suited to synthesized client-side tools: generated code can be compiled into an artifact that the browser can host, while the surrounding system controls UI, file access, and result presentation.

Browser-local file access. MoonAP relies on browser file picker and file system capabilities for local workflows. The File System Access API is developed through the Web Platform Incubator Community Group, and browser compatibility remains narrower than baseline web APIs [7, 4]. For this reason, v0.1 validates large-file workflows primarily in Chrome and Edge rather than claiming universal browser support.

Safety and artifact reuse. Generated code can introduce security and provenance risks, as prior studies of AI-assisted code generation have emphasized [6]. MoonAP v0.1 does not solve this problem fully. Its contribution is an artifact structure and execution boundary that make generated applications inspectable and reusable, while leaving stronger provenance, signing, sandboxing, and dependency controls as future work.

3 Motivation and Design Goals

MoonAP began from a concrete need: a user should be able to ask for a small application, run it immediately, and keep it if it works. The system was also motivated by a privacy-sensitive large-file case study: generating and analyzing FastQ sequencing-like files in the browser without uploading file contents to an LLM. Over time, the project evolved from a domain-specific demonstration into a general agent playground.

The design goals of v0.1 are as follows.

MoonBit-first synthesis. The target language of generated applications is MoonBit. The system should keep MoonBit source, compiler output, WebAssembly artifacts, and runtime metadata visible enough to support debugging and reuse.

Browser-local execution. Generated applications should run in the user's browser where possible. This is especially important for file-oriented workflows, where local files should not be sent to the LLM or to a remote execution service.

Reusable artifacts. A successful run should not be a one-off event. The generated application should be saved as a structured package that can be reopened and executed later. Reuse turns a generated answer into a local software asset.

Local data and reduced repeated LLM calls. Large files should stay near the user, and repeated executions should not require repeated LLM calls. This principle is important for privacy, latency, cost, and reliability: the cloud model should help synthesize the tool, while the browser should do ordinary work with the user's data.

Separation of generation and runtime. The LLM can generate source code and runtime metadata, but the browser host should control file pickers, local storage, large-file streaming, and presentation of results.

Stable demonstration path. Because real LLM providers vary in availability and MoonBit capability, the system should include a deterministic simulated route for repeatable demonstrations and regression checks.

4 System Overview

Figure 1 shows the MoonAP pipeline. A browser UI accepts a natural-language task. The local server routes the request to a real LLM provider or a simulated LLM path. The resulting MoonBit source is compiled to WebAssembly using the MoonBit toolchain. The browser then renders a runtime UI from the artifact's runtime contract, executes the browser-local step, and records the result. If the user accepts the result, the app is materialized as a reusable SKILL.

The pipeline is deliberately asymmetric. Prompts and generated source may cross the LLM/server boundary during synthesis, but user-selected data files are handled by browser-local runtime code. This keeps high-volume and privacy-sensitive data out of the model call, while still allowing small summaries, result metadata, and saved runtime specifications to be recorded for reuse.

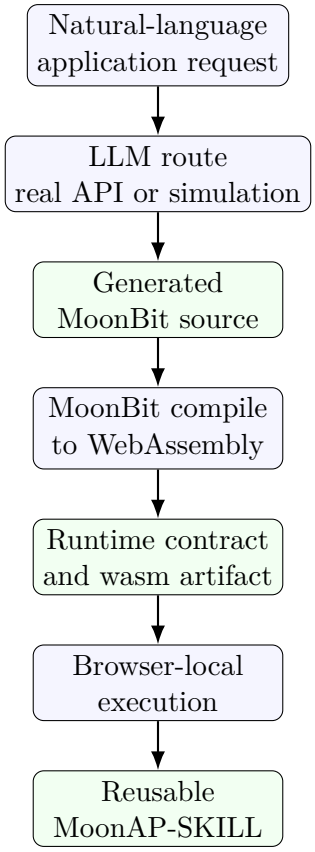

Figure 1: End-to-end MoonAP workflow. The LLM participates in source generation, while browser-local execution and artifact reuse are controlled by the host system. Large user data files stay on the browser side of the workflow.

5 MoonAP-SKILL Architecture

MoonAP-SKILLs are reusable application packages. A saved SKILL contains a human-readable `SKILL.md`, generated MoonBit source, a compiled WebAssembly artifact, runtime metadata, and optional distribution files. The design goal is to make a generated application behave less like a transient chat response and more like a local tool that can be inspected, copied, installed, and rerun. The v0.1 design separates SKILLs into three layers (Figure 3).

This separation addresses three practical issues. First, it prevents cloud catalog entries from being treated as directly executable objects before they have been installed and inspected locally. Second, it gives the user a clear distinction between self-created artifacts and public artifacts obtained from a hub. Third, it separates discovery from execution: the cloud hub can advertise and distribute packages, while actual runs happen from local stores under the user's control. Figure 4 shows the browser UI for selecting a Local-SKILL-Hub directory, inspecting installed SKILLs, refreshing the Cloud-SKILL-Hub catalog, installing public SKILLs, and running local SKILLs.

6 Runtime Contracts

One of the central lessons from v0.1 is that synthesized source code is not enough to produce a usable application. The host must know what input controls to render, whether an input file is required, how results should be displayed, and whether the output should be downloaded, shown as a report, or treated as an interactive view.

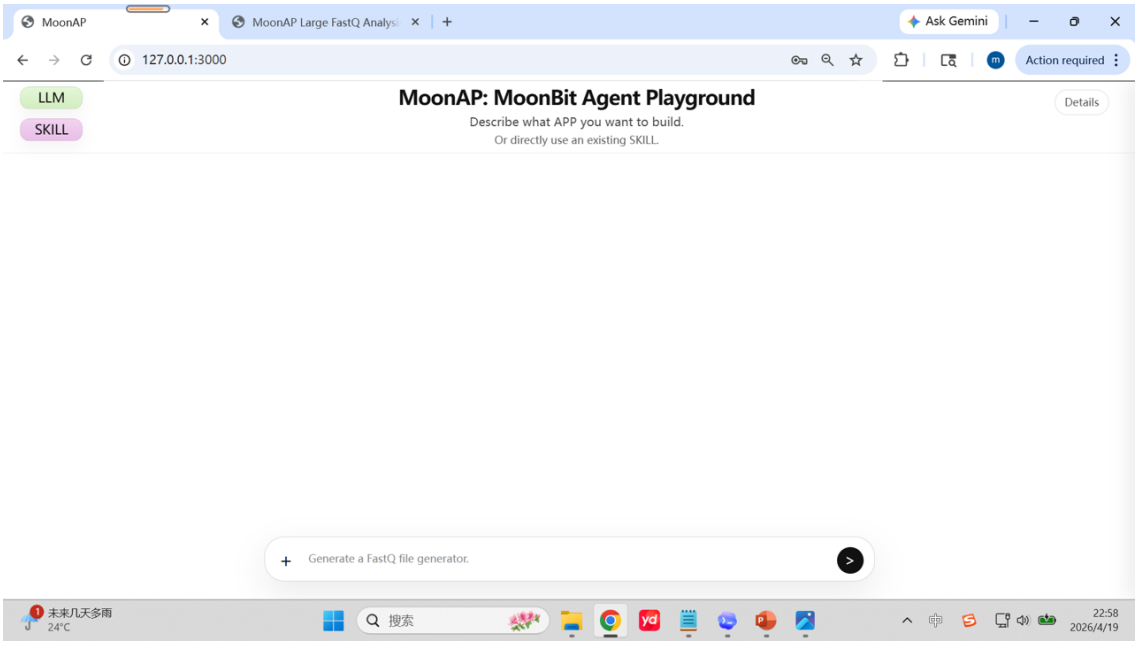

Figure 2: MoonAP browser UI. Users can enter a natural-language application request or switch to the SKILL interface for reuse and installation.

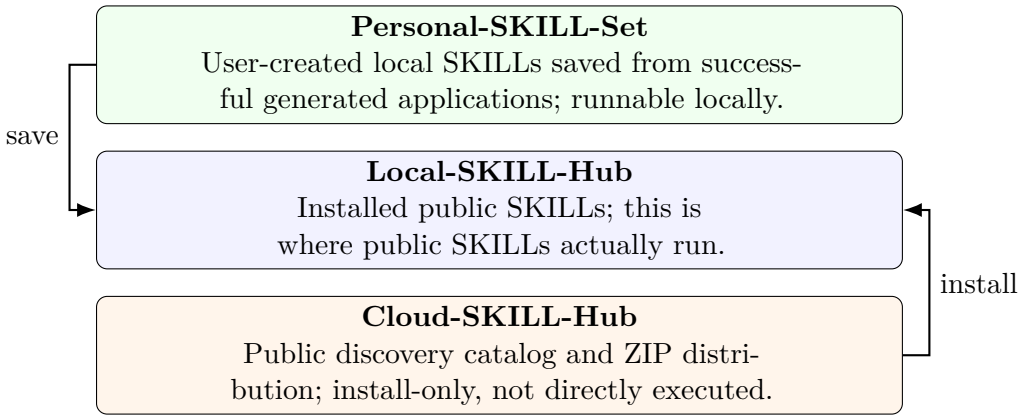

Figure 3: Three-layer MoonAP-SKILL model. Cloud entries are used for discovery and installation; execution happens in Personal or Local stores.

MoonAP uses runtime contracts for this purpose. A runtime contract is a structured specification associated with a generated artifact. The contract is intentionally host-facing: it tells the browser what experience to provide around the compiled program. It can include:

- runtime mode, such as `form`, `file`, or `interactive`;
- result mode, such as `text`, `report`, or `download`;
- input fields with labels, types, default values, and validation hints;
- an I/O contract for file or large-file workflows;
- action labels and rerun labels for the UI;
- tool-specific metadata, such as CSV summary or FastQ analysis.

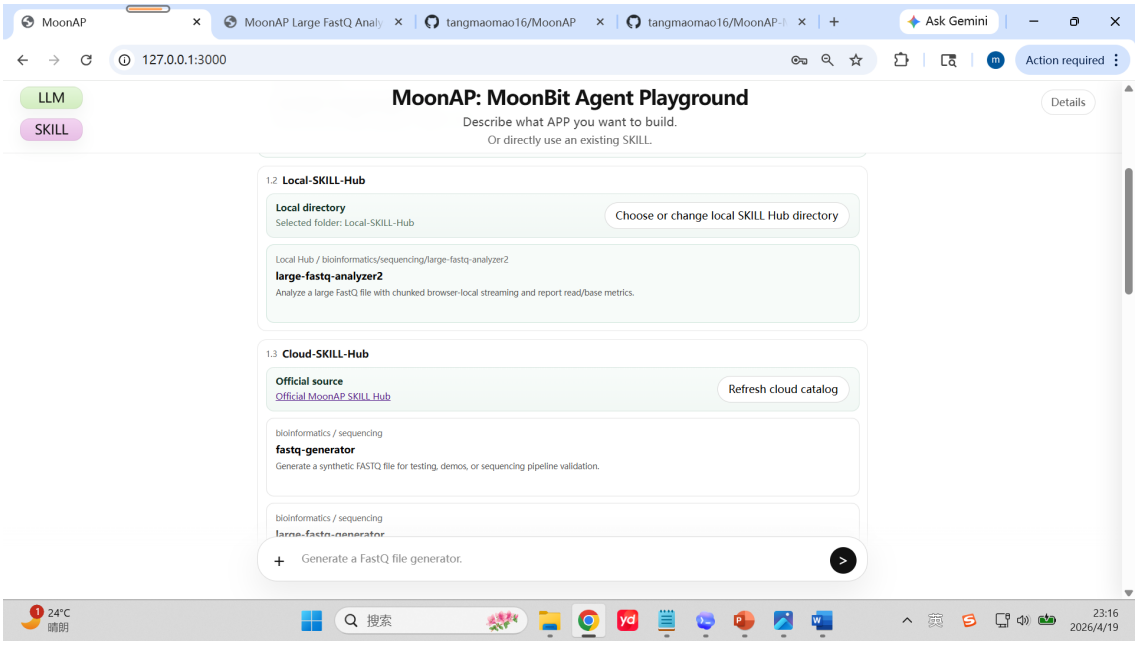

Figure 4: MoonAP Local-SKILL-Hub and Cloud-SKILL-Hub interface. Installed public SKILLs run from the local hub, while cloud entries are used for discovery and installation.

The runtime contract enables generic execution paths for many small tools. For example, the Celsius/Fahrenheit converter, minutes/seconds converter, BMI calculator, circle calculator, loan payment calculator, tip calculator, JSON formatter, and text analyzer can all be represented as form or text contracts. The CSV analyzer uses a file contract with a browser-local file picker and report output. Without this contract layer, each generated app would require bespoke front-end glue code or manual intervention before it could feel like a usable application.

The contract also makes SKILL reuse possible. When a SKILL is run again, MoonAP does not need to ask the LLM to regenerate code. Instead, it reloads the saved runtime specification, renders the appropriate UI, and executes the local runtime path. This keeps the LLM call focused on creation and repair, not on every future use of the tool.

7 Browser-Local Large-File Workflows

MoonAP v0.1 includes two large-file FastQ workflows: generation and analysis. These workflows are intentionally treated as high-value demonstrations of browser-local computation rather than as the only purpose of the platform. They also illustrate an important design point for LLM-generated applications: the LLM should help create the tool, but it should not need to ingest the user's raw data file.

Large FastQ generation. The generator uses a browser file-save picker and a streamed writer. The user chooses an output file, enters parameters such as target size, read length, random seed, and N-base rate, and the browser writes records locally. This design can produce large synthetic files, including 1GB-scale demonstrations, without constructing the whole file in server memory. The generated file contents are not sent to the LLM.

Large FastQ analysis. The analyzer uses a browser file-open picker and reads the selected file in chunks. It counts reads, lines, bases, base composition, malformed records, read lengths, and elapsed time. Because the raw bytes are processed through browser-local chunking, the workflow

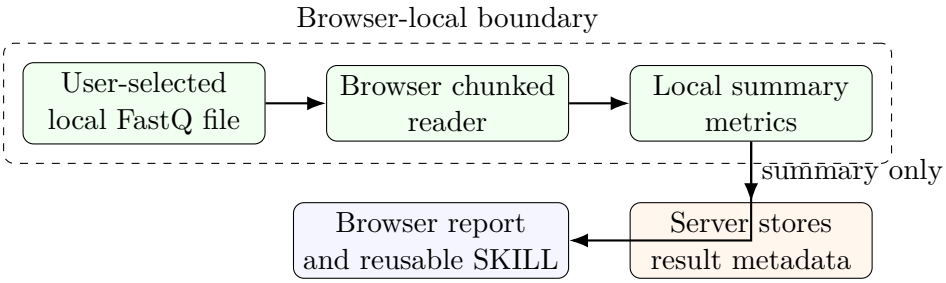

Figure 5: Browser-local large-file analysis. File contents are processed in the browser; only summary metadata is recorded by the local server.

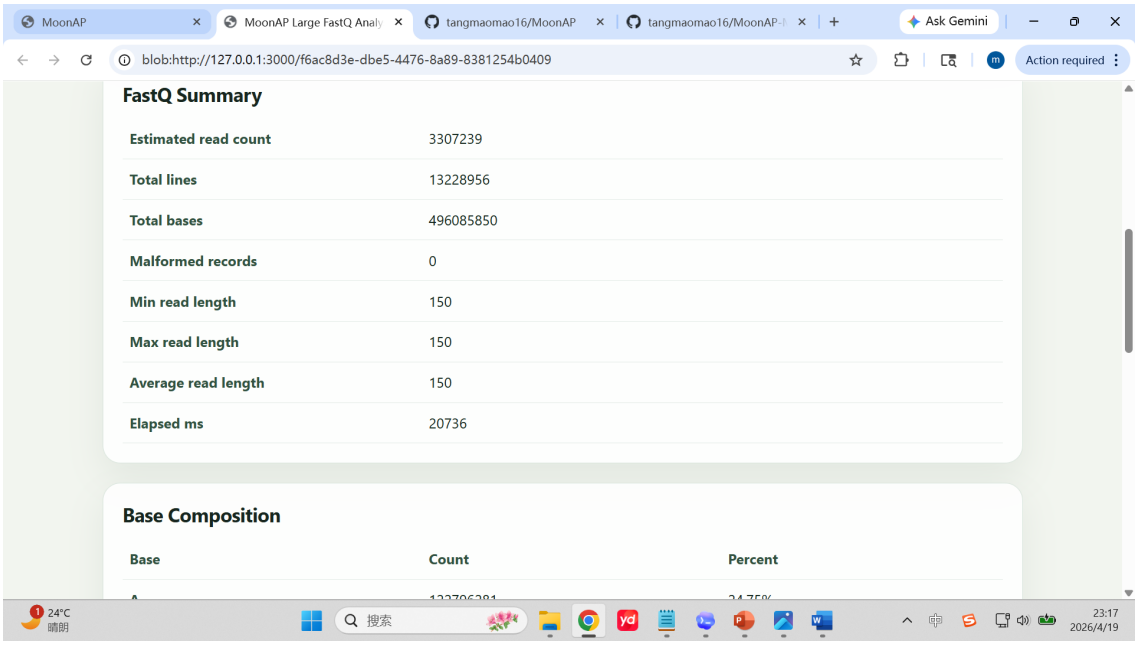

Figure 6: FastQ analysis report with summary metrics and the beginning of the base-composition table. The counts are computed locally in the browser from the selected FastQ file.

is suitable for large local files, including GB-scale demonstrations on commodity machines. File contents stay in the browser; the server receives only summary metrics and result metadata.

These workflows are important because they exercise a workload that ordinary chat-based code generation does not handle well. Sending a GB-scale data file to an LLM would be expensive, slow, and often impossible under token limits. In MoonAP, the prompt and tool specification can be small, while the high-volume data path remains local.

Figure 6 shows the browser-local FastQ report generated by the analyzer. The report is opened from a browser blob URL and presents summary metrics and base-composition results without uploading the original file contents.

8 Implementation

The prototype is organized as a MoonBit project with a local server package and a browser application package. The server handles HTTP endpoints, LLM routing, runtime registration, artifact persistence, SKILL export, and static file serving. The browser app implements the user interface, runtime cards, file pickers, generic runtime execution, large-file progress reporting, SKILL installation, and SKILL reuse.

The system currently targets a Windows-first release path. Native server builds use the MoonBit toolchain together with MSVC setup scripts. The browser side is compiled to JavaScript and committed as distribution bundles for fresh-clone startup fallback. The primary validated browser target is Chrome or Edge, because large-file workflows depend on the File System Access API [7].

8.1 LLM Routes

MoonAP separates real and simulated LLM routes. Real routes support multiple providers or OpenAI-compatible endpoints. The simulated route provides deterministic outputs for the v0.1 demo set. This distinction is important because real lightweight models often fail to produce valid MoonBit in the current ecosystem. The simulated route is not presented as an evaluation of model quality; it is a system-testing path that exercises the compile, runtime, and SKILL reuse pipeline.

This choice keeps two claims separate. MoonAP does not claim that every current LLM can reliably generate MoonBit. It claims that the surrounding platform can turn a successful or simulated generation into a compiled, runnable, reusable browser-local application. As MoonBit becomes more common in model training data and stronger providers improve, the same platform path can absorb better generation quality without changing the runtime architecture.

8.2 Artifact Lifecycle

The lifecycle of an artifact has four stages:

1. **Generation:** prompt and LLM output are captured.
2. **Compilation:** MoonBit source is compiled into a WebAssembly artifact.
3. **Runtime:** the browser executes a contract-driven runtime step and records a result.
4. **Reuse:** the artifact is materialized as a SKILL and later run from Personal-SKILL-Set or Local-SKILL-Hub.

This lifecycle allows MoonAP to treat a successful generated app as an asset rather than a transient response.

9 Evaluation and Case Studies

The v0.1 evaluation is qualitative and prototype-oriented. It asks whether the system can express, run, and reuse representative browser-local applications. Table 1 summarizes the current demonstration matrix.

The evaluation is organized around platform capability rather than model leaderboard accuracy. A demonstration category is considered successful when the system can produce or load the application, expose the correct runtime controls, run the browser-local workflow, present the result, and preserve the artifact for later reuse where applicable.

The small utility tasks exercise the generic contract path. They show that the browser can render task-specific controls without bespoke UI code for each generated app. The CSV analyzer extends this pattern to file input and report output. The FastQ generator and analyzer exercise specialized large-file runtime paths that rely on browser-local streaming and chunking. Together, the cases cover both ordinary utility apps and data-heavy local workflows.

The most important engineering result is the reuse path. A successful generated app can be saved as a SKILL, reopened, and executed from a local store without another LLM call. This substantially changes the user experience: the LLM is used to create or repair an artifact, but the artifact can later behave like a normal local application.

Capability family	Examples	Runtime path
Computed form apps	Celsius/Fahrenheit, minutes/seconds, two-number sum, BMI, circle, loan, tip	Generic form contract
Text and JSON utilities	JSON formatter/validator, text analyzer	Generic text runtime
Browser-local file utility	CSV summary analyzer	Generic file/report runtime
Large-file generation	Large FastQ generator	Streamed browser writer
Large-file analysis	Large FastQ analyzer	Chunked browser reader/report
Reuse	Personal SKILL, Local-SKILL-Hub	Saved runtime spec
Sharing/install	Cloud-SKILL-Hub	Catalog and ZIP install

Table 1: MoonAP v0.1 demonstration matrix.

10 Lessons Learned

Generated code needs a host contract. Without a runtime contract, the browser does not know which inputs to show or how to interpret results. Embedding or saving this contract makes generated apps reusable.

A deterministic path is necessary for system development. When the target language is young, real models may fail for reasons unrelated to the runtime system. The simulated route allowed v0.1 to test compilation, UI, file handling, and SKILL reuse deterministically.

Reuse is as important as generation. Users should not need to pay the latency and uncertainty cost of an LLM call every time they run a useful generated app. The SKILL format turns synthesis into a reusable software artifact. This also reduces repeated cloud LLM usage: the generative step is used for creation or repair, while repeated execution happens locally from the saved contract and compiled artifact.

Browser-local file boundaries must be explicit. For large files, privacy and performance depend on a clear boundary: file contents stay in the browser, while summaries and metadata can be recorded by the local server.

Client hardware is part of the execution platform. Browser-local WebAssembly execution lets MoonAP use computation and storage resources already available on the user's device. This edge-side execution model is especially valuable for file utilities, where moving raw data to a cloud model would be slower, more expensive, and less private than processing it near the data.

11 Limitations

MoonAP v0.1 has several limitations. First, real LLM generation of valid MoonBit is not yet reliable across lightweight or free models. Stronger models may perform better, but model capability and provider availability are external variables. Second, the current release path is best tested on Windows with Chrome or Edge; the File System Access API is not a universal browser baseline [4]. Third, the generic runtime contract is still small and does not yet cover arbitrary application UIs, complex stateful apps, or a universal WebAssembly host ABI. Fourth, the evaluation is a prototype case study rather than a large-scale benchmark. Finally, security hardening remains future work: generated code, saved artifacts, and hub-installed packages need

stronger provenance and sandboxing mechanisms before broad deployment, a concern consistent with prior work on the security risks of generated code [6].

12 Future Work

Future versions of MoonAP can extend the platform in several directions:

- a more formal runtime contract schema and validation layer;
- a stable host ABI for generated WebAssembly programs with file, stream, and UI callbacks;
- stronger automated repair loops for MoonBit compilation errors;
- cross-platform server support beyond the current Windows-first path;
- richer SKILL provenance, signatures, versioning, and dependency management;
- quantitative evaluation over larger prompt suites and real model providers;
- improved UI generation for interactive applications and games.

13 Conclusion

MoonAP v0.1 demonstrates an end-to-end MoonBit software synthesis workflow: natural-language task descriptions become MoonBit source, compiled WebAssembly artifacts, browser-local applications, and reusable SKILL packages. The system highlights a practical lesson for LLM-based software synthesis: generation is only the first step. A usable platform also needs runtime contracts, local execution boundaries, artifact packaging, deterministic testing paths, and reuse mechanisms. By keeping raw data on the client, using browser-side computation for file workflows, and reusing saved SKILLs without repeated generation, MoonAP points toward a more privacy-preserving and cost-aware style of practical software synthesis. The v0.1 prototype is therefore not merely a FastQ demo or a code-generation wrapper; it is an application lifecycle platform for turning generated MoonBit programs into local tools. Although current lightweight LLMs do not yet generate MoonBit reliably, MoonAP shows how a MoonBit-centered agent playground can make generated applications inspectable, runnable, reusable, and data-local in the browser.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. In *NeurIPS Workshop on Machine Learning for Programming*, 2021. arXiv:2108.07732.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie

Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi:10.1126/science.abq1158.

[4] MDN Web Docs. File System API. https://developer.mozilla.org/en-US/docs/Web/API/File_System_API. Accessed 2026-04-21.

[5] MoonBit Team. MoonBit documentation. https://docs.moonbitlang.com/. Accessed 2026-04-21.

[6] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768, 2022. doi:10.1109/SP46214.2022.9833571.

[7] Web Platform Incubator Community Group. File System Access API. https://wicg.github.io/file-system-access/. Accessed 2026-04-21.

[8] World Wide Web Consortium. WebAssembly Core Specification. https://www.w3.org/TR/wasm-core-2/. Accessed 2026-04-21.